

Leverage Your Programming Skills with Erlang!

The Erlang programming language is used to build massive scalable real-time software systems that require high availability. Some of its uses are in telecom, banking, e-commerce, computer telephony and instant messaging. The runtime system of Erlang has built-in support for concurrency, distribution and fault tolerance. Read on to get more familiar with it.

The first time I programmed with a functional programming language was back in 1996 while doing my MSc at UCL (University College London). The language was called SML or Standard ML. Like all functional programming languages, a key feature of SML is the *function*, which is used for abstraction. Nowadays, functional programming languages like Erlang are back.

Joe Armstrong created Erlang at the Ericsson Computer Science Laboratory. At the time of writing this article, the latest Erlang version is *R16B*.

Installing and running Erlang

Your Linux/UNIX distribution probably includes a ready-to-install Erlang package; so go ahead and install it. After doing so, try running the `erl` command which should give you the following output:

```
$ erl
Erlang R16B (erts-5.10.1) [source] [64-bit] [smp:2:2] [async-threads:10] [hipe] [kernel-poll:false]
Eshell V5.10.1 (abort with ^G)
1>
```

If the output you get is similar, then Erlang has been successfully installed; so try to type the following commands inside the Erlang shell to get to know more about it:

```
1> pwd().
/Users/mtsouk/docs/article/Erlang.OSFY
ok
2> cd("../").
/Users/mtsouk/docs/article
```

```
ok
3> 5-4.
1
4> 10*v(3).
10
5> math:pi().
3.141592653589793
6> q().
ok
7> mtsouk$
```

Now, let's look at each executed command:

- Command 1: The `pwd()` command prints the current working directory. You can understand that each command ends with a dot – ‘.’. The output is presented after typing the ‘.’ and pressing the *Return* key.
- Command 2: The second command changes the working directory and prints the new directory.
- Command 3: This subtracts two integer numbers and prints the result.
- Command 4: Now a tricky thing happens. We multiply the result of the third command, represented by `v(3)`, by 10.